

目 录

1	问题描述.....	2
1.1	游戏背景.....	2
1.2	需要解决的问题.....	3
2	系统设计.....	3
2.1	功能模块设计.....	3
2.2	数据结构设计.....	5
3	详细设计.....	6
3.1	玩法设计.....	6
3.2	美术设计.....	6
3.3	剧情设计.....	14
3.4	音乐设计.....	14
3.5	代码设计.....	15
3.5.1	图像输出.....	15
3.5.2	更新区块.....	15
3.5.3	获取操作.....	16
3.5.4	粒子效果.....	17
3.5.5	障碍物移动.....	18
3.5.6	障碍物碰撞逻辑.....	18
3.5.7	记图变量.....	19
3.5.8	小恐龙重力系统更新的实现.....	19

小恐龙跑酷电脑游戏软件

摘要：本作品是一个操控小恐龙躲避障碍的跑酷游戏，在跑酷游戏逻辑的基础上，提高了画面的精美度。用户操控小恐龙躲避灾难，在这个时空中，使小恐龙逃离灭绝。但是随着小恐龙活到了人类时代，因为人类的大肆破坏，他们的家园又被灭绝。最终，小恐龙还是成为了历史的记忆

关键词：跑酷;游戏;保护地球;

1 问题描述

1.1 游戏背景

在史前时代的某个角落，有一只可爱的小恐龙，它的家园正受到巨大的威胁。原本安宁的恐龙世界因为一场灾难而面临灭绝的危机。火山爆发、地震和陨石撞击将整个地区变成了一片混乱和危险的地带。

小恐龙必须勇敢地踏上逃生之旅，通过各种考验和障碍来寻求生存。在逃离的过程中，它将经历炎炎沙漠、灼灼火山，最终到达归家之路。

在炎炎沙漠中，荒凉的环境和带刺的仙人掌将成为小恐龙行进的障碍。在灼灼火山，炽热的岩浆和从天而降高速坠落的陨石威胁着它的安全，小恐龙必须快速穿越，避开岩浆喷发和坠落的陨石。在最后的归家之路中，看似静谧的平原上不断涌现的高耸大树将考验它的反应和跳跃技巧。

然而，小恐龙并不孤单。它遇到了你，你将和它一起面对挑战并携手度过危机。

在这个逃生的游戏中，玩家将操作小恐龙，帮助它跳跃、奔跑、躲避障碍物，尽可能远离灾难，找到新的安全之地。通过遇到新的时空裂缝，玩家可以帮助小恐龙穿越到新的场景，让它不断到达安全之地。

尽管面临着灭绝的威胁，小恐龙将与玩家一起展开冒险旅程，带来刺激和紧张的游戏体验。只有通过勇气和智慧，才能帮助小恐龙逃离这个即将毁

灭的世界，找到新的生存希望。

1.2 需要解决的问题

- (1) 实现小恐龙的生成，移动，以及操控的逻辑
- (2) 实现屏幕范围内的多个障碍物生成，并同时移动
- (3) 实现背景和地面的差速移动
- (4) 实现鼠标操控按键，并且不影响键盘的敲击
- (5) 实现多个场景的转换，并且每个场景有对应的一组障碍物，粒子效果
- (6) 生成随机粒子增加画面效果，对于每个画面都生成不同风格颜色的粒子
- (7) 实现自由音效，每个操作都有音效，并且还有背景音乐

2 系统设计

2.1 功能模块设计

模块设计分为页面，小恐龙，障碍物三个板块，各个板块分工明确，命名讲究，组成的部分严谨。

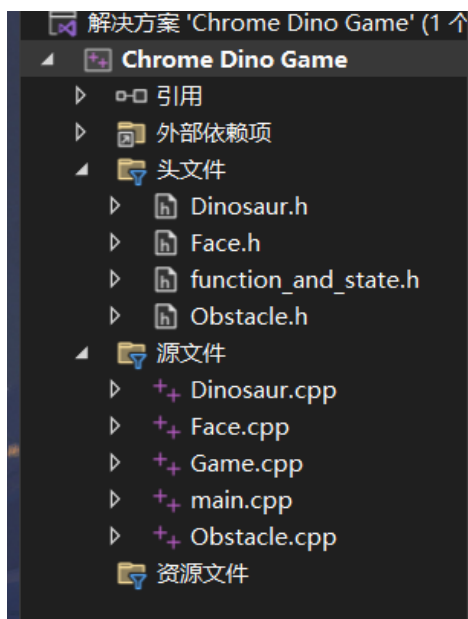


图 1 各功能模块分工展示

各个功能模块设计如图 2 所示

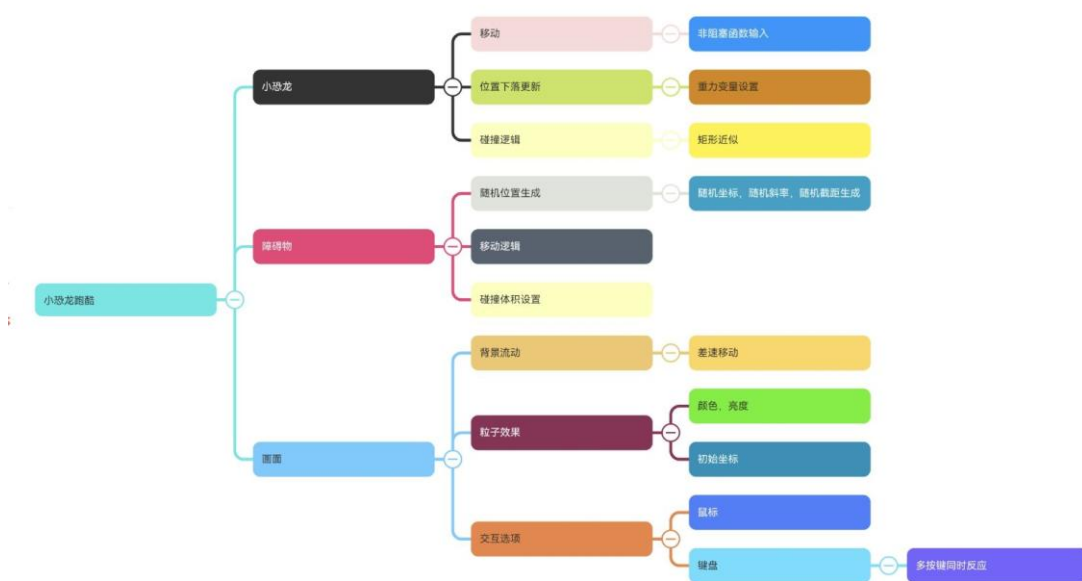


图 2 系统功能模块图

其中 face 模块实现各种页面的展现，包括主页，暂停页面，死亡页面，游戏背景介绍，以及颗粒效果绘制，流动背景绘制，按钮绘制。

Dinosaur 模块实现关于小恐龙的各种逻辑操作，包括移动，碰撞判定，法阵操作，移速判定，场景转换判定。

Obstacle 模块实现障碍物生成，障碍物移动。障碍物包括陨石和植物，陨石具有随机斜率生成和随机截距生成。

2.2 数据结构设计

有着详细的宏定义，如下图举例一个植物块的宏定义：

```
#define PLANT_COUNT 2// 定义一组植物数量
#define Plant_height 100
#define Plant_width 65
#define New_Plant_width 50
#define New_Plant_height 50
#define New_Plant_X 300
#define New_Plant_Y 300

#define New_Plant_Frequency 2
//更改转换场景的刷新频率
```

图 3 植物块宏定义

为微粒定义了微粒结构体

```
extern struct ASH {
    double x;//x坐标
    int y;//y坐标
    double step;//移动速度
    int color;//颜色
};
```

图 4 微粒结构体

为按钮定义按钮结构体

```
extern struct Button {
    int x, y, width, height;
    COLORREF inColor, outColor, curColor;
    char* text;
};
```

图 5 按钮结构体

为陨石定义陨石坐标结构体和斜率截距全局变量

```
extern double slope[METEORITE_COUNT]; //斜率数组
extern double nodal[METEORITE_COUNT]; //截距数组

extern struct Meteorite {
    int x;
    int y;
};
```

图 6 陨石结构体

3 详细设计

3.1 玩法设计

(1) 核心玩法：用键盘操控小恐龙躲避障碍物

(2) 胜利条件：共有三关，每一关都有不同的障碍物。必须躲避每一个障碍物，进入每一关的传送门，才能到达下一关。

(3) 障碍物种类： 仙人掌：从右向左移动

椰树：从右向左移动，但稍矮

陨石：从右上到左下移动，有不同的斜率和截距

(4) 难度梯度：游戏整体会逐渐增加移速（每 5 秒增加一次，增加 5 次达到上限），难度会不断加大

3.2 美术设计

(1) 主页设计

主页的整体风格是黄沙漫天，预示着小恐龙将遇到灾难。开头对游戏的整体氛围渲染很重要。

主页的背后有黄沙粒子移动，每一颗都有单独的颜色和单独的速度。

主页后的背景也在流动，与黄沙共同作用下主页显得不失活力

将鼠标放到按钮上，按钮的底色会发生变化，文字的颜色也会发生变化，文

字的颜色是完全随机的，即可出现任何颜色变化



图 7 主页图

(2) 游戏背景场景设计

此处的背景也是流动的，月光、背后的山峰和白云预示着这片土地的宁静

月光照亮会小恐龙前方的脚步，点亮它在黑暗中的归途。月光所描绘的道路象征着安全。

粒子也是流动的，但此处的粒子效果不同于之前的黄沙漫天的效果，这里的粒子色彩更加斑斓，这也是游戏的第三个场景，预示着归家之路的美好。与开头介绍的灾难截然相反。营造出神秘而温暖的氛围。



图 8 背景介绍图

(3) 游戏实际效果

此处背景、地面和粒子效果与小恐龙也是差速移动的。

背后的粒子，每一颗都有单独的颜色，单独的速度。而且他们的颜色并不是完全随机，只是在区域内随机。在研究了 RGB 图之后，最终找到了合适的艳红色系的 RGB 取值区间。实际生成的粒子就好像从岩浆中蹦出来的火星子一样。呈现的效果仿佛身临其境。

第一个场景的背景是一片沙漠，障碍物是仙人掌。

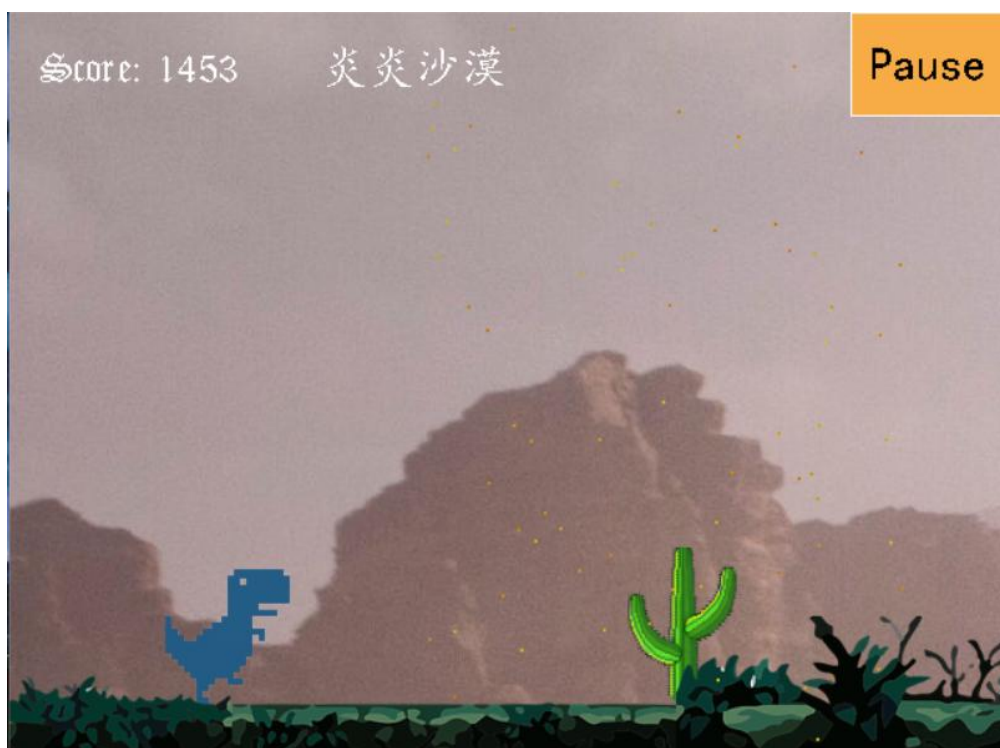


图9 第一个场景地图

第二个场景的背景是一座冒着岩浆的火山。寻找背景的时候，对这个场景的表现力需求极高，因为这个场景是小恐龙最危险的场景。这个火山要喷发、冒着光，背景还要是黑偏红。

障碍物是陨石，陨石在这里起到了画龙点睛的作用，火山配陨石十分贴合。这样的场景融合在一起，整个场景显得活灵活现，充满表现力。



图 10 第二个场景地图

第三个场景是夜晚，障碍物是树。



图 11 第三个场景地图

死亡效果是将小恐龙坐标代入二次函数，营造出小恐龙被打肉的击退效果。这样呈现的好处是让玩家有充分的反馈，使游戏呈现更加真实。

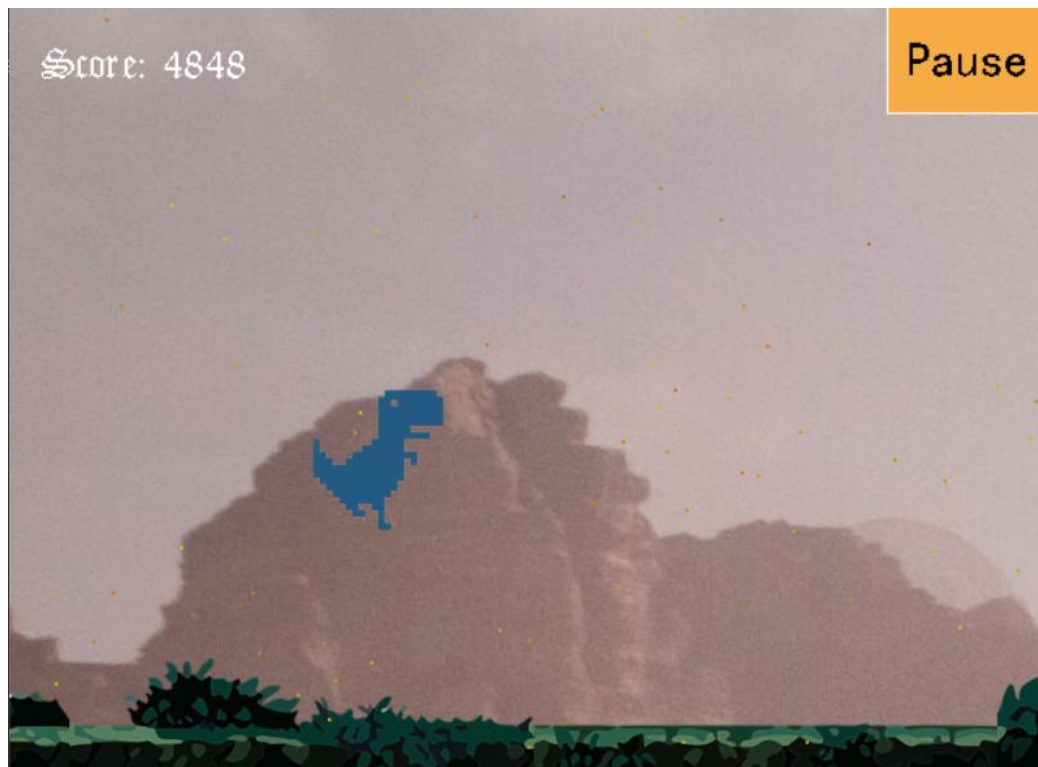


图 12 死亡状态瞬间

右上角的暂停按钮，用鼠标点击可以暂停。



图 13 暂停界面

点击“继续游戏”，将会继续游戏；点击“返回主页”，将会返回如图二所示的主页；点击“游戏按键”，会出现游戏按键教程，如下图所示：



图 14 游戏按键界面

(4) 障碍物设计

有仙人掌，椰树，陨石。

其中，仙人掌是第一个沙漠的场景。仙人掌是沙漠一贯的特色。将仙人掌置于沙漠之中，暗含着危机，又让原本荒凉的沙漠增添了一分活力。

陨石是第二火山的场景。同样是热气灼灼的效果，陨石和火山带来的冲击感是灾难中最强烈的，陨石落到地上还有炸裂的音效和图片，使灾难看起来更加活灵活现。配上背景和粒子效果，这一关的色彩呈现是最精彩的。

大树是第三个夜晚的场景。作为最安静、看起来最安全的障碍物，其实也暗含杀机，在最后的地图中，也是小恐龙最接近安全的地方。这里的设计整体偏向寂静、美好，所以大树作为这一关的障碍物能和环境完美的容纳进去。

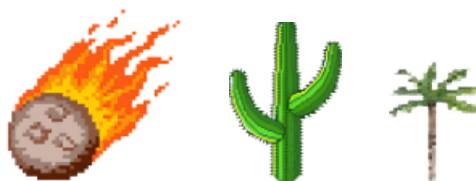


图 15 三种障碍物展示

3.3 剧情设计

在平行世界的史前时代的某个角落，有一只可爱的小恐龙，它的家园正受到巨大的威胁。原本安宁的恐龙世界因为一场灾难而面临灭绝的危机。火山爆发、地震和陨石撞击将整个地区变成了一片混乱和危险的地带。

小恐龙必须勇敢地踏上逃生之旅，通过各种考验和障碍来寻求生存。在逃离的过程中，它将经历炎炎沙漠、灼灼火山，最终到达归家之路。

总共有三个地图，对应三个不同的关卡，伴随关卡的转换在上方也会有文字进行提示

小恐龙如果成功逃脱，就会迎来游戏的结局：逃离了自然灾害，存活到了人类时代。但随着人类对自然环境的破坏，小恐龙最终因为家园的再度破坏，最终也没有逃离灭绝的命运。用结尾衔接开头，引发人们对保护自然环境的思考。

3.4 音乐设计

游戏中采用不同的音效，配合背景音乐，起到了身临其境的效果，包括：

1. 背景音乐选的是轻松优雅的旋律，搭配这款治愈风的游戏
2. 主页面，暂停界面等能用鼠标点击的时候均有玻璃破碎的清新音效，给整体的氛围营造一种锐利的感觉。
3. 陨石落地的音效，赋予陨石强烈的打击感
4. 小恐龙跳跃的音效，使游戏增添活力感

3.5 代码设计

代码框架为 while(1) 输出，更新，获取操作；

3.5.1 图像输出

输出部分采用双缓冲的逐帧输出，让画面稳定

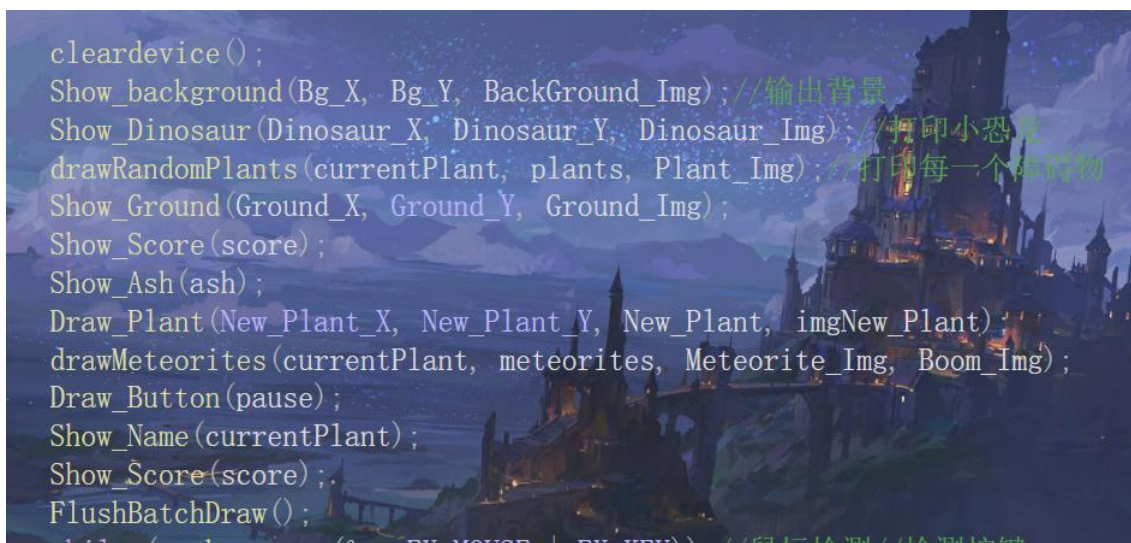


图 16 双缓冲输出模式

3.5.2 更新区块

为了实现各个部分的差速移动，也就是每个部分的位置更新速度是不一致的，我们采用了静态全局函数——计时器。计时器可以检测每一个序号的持续时间经过的时间跨度，从而判断是否更新这个部分的坐标。



图 17 计时器更新坐标

3.5.3 获取操作

作为一个跑酷游戏，对于按键的反应要更加灵敏迅速。本程序基于非阻塞信息获取函数 `peekmessage()` 实现，同时获取键盘鼠标信息，并且获取速度快，可实现多个按键同时按下的反应操作。给用户更加良好的运行体验。

下图为键盘获取模块

```
while (peekmessage(&m, EX_MOUSE | EX_KEY)); //鼠标检测//检测按键
if (m.message == 256) //按键按下
{
    switch (m.vkcode) {
        case VK_UP:
        case 'W': w = true; break;
        case VK_DOWN:
        case 'S': s = true; break;
        case VK_LEFT:
        case 'A': a = true; break;
        case VK_RIGHT:
        case 'D': d = true; break;
    }
}
if (m.message == 257) //按键抬起
{
    switch (m.vkcode) {
        case VK_UP:
        case 'W': w = false; break;
        case VK_DOWN:
        case 'S': s = false; break;
        case VK_LEFT:
        case 'A': a = false; break;
        case VK_RIGHT:
        case 'D': d = false; break;
    }
}
```

图 18 键盘获取模块

下图为鼠标获取模块（运行中的暂停板块）

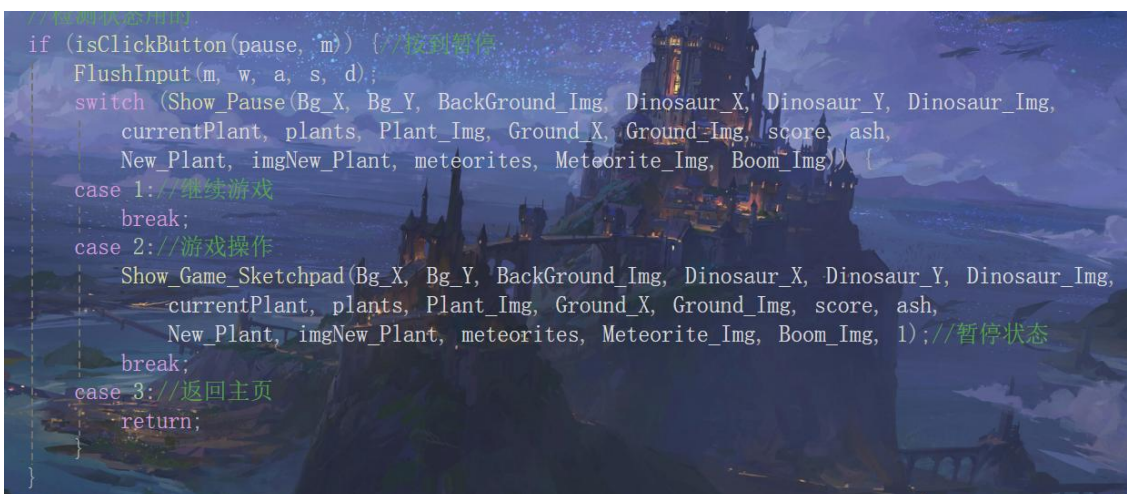


图 19 鼠标获取模块

3.5.4 粒子效果

一个粒子其实是四个像素点（因为一个像素点很难观察到），利用随机函数生成每一个粒子的坐标和颜色。

不同风格颜色和坐标的粒子生成如下图所示（体现在 RGB 的范围上）

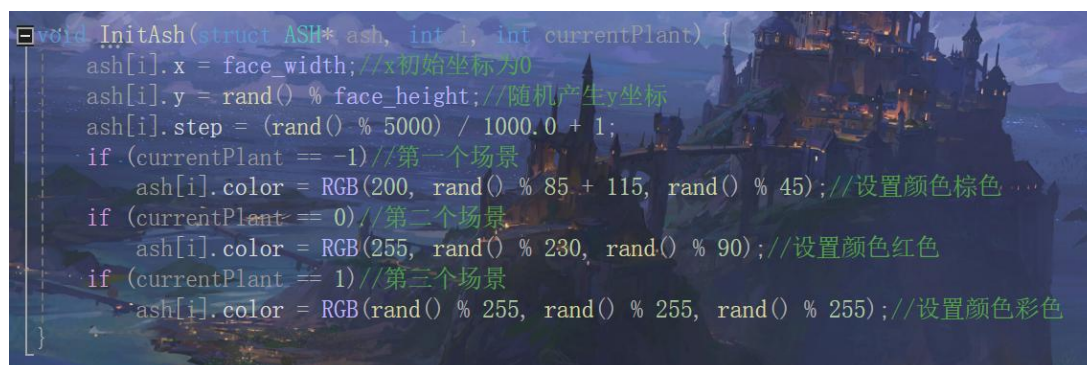


图 20 初始化粒子函数

打印粒子其实是打印四个像素点，用 putpixel 实现高性能输出

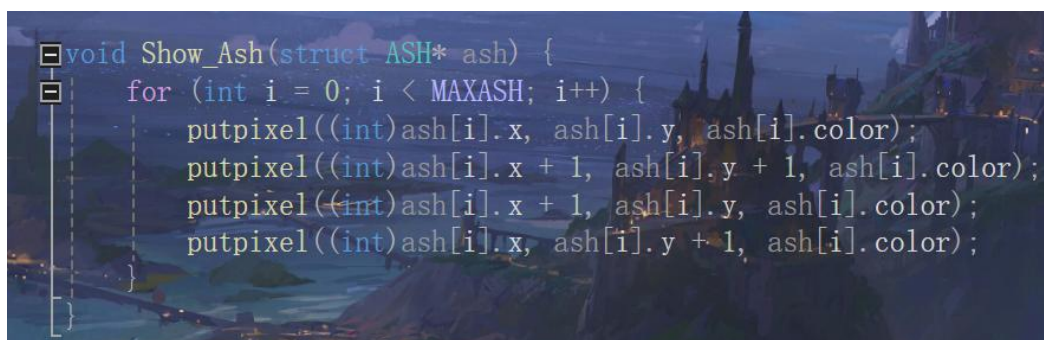


图 21 输出粒子函数

3.5.5 障碍物移动

障碍物的生成是从地图右侧以外生成，逐渐改变坐标移动到屏幕内出现，则视为可见。障碍物的生成采用了随机函数。对于植物来说，植物之间的水平间距是随机生成的；对于陨石来说，陨石具有初始坐标，下落斜率，所以陨石需要初始化斜率、截距和初始坐标。下图为陨石的初始化函数：

```
void initializeMeteorite(Meteorite *meteorites, double* slope) //初始化陨石
{
    meteorites[0].x = 640 + rand() % 300; //右上角
    meteorites[0].y = 0;

    for (int i = 0; i < METEORITE_COUNT; ++i) //随机斜率
    {
        slope[i] = -1 + ((double)rand() / RAND_MAX) * (2 - 1); //生成 1 到 2 之间的随机浮点数
    }

    for (int i = 1; i < METEORITE_COUNT; ++i) //随机x, y坐标
    {
        meteorites[i].x = meteorites[i - 1].x + rand() % 300 + 110;
        meteorites[i].y = meteorites[i - 1].y + 480;
    }
}
```

图 22 陨石初始化函数

注：陨石的一组刷新数量是宏定义的，为了平衡难度，陨石的一组刷新数量为 1，所以图代码中第二个 for 循环被警告没有运行，但这也是代码完整性和易于修改性的体现。

3.5.6 障碍物碰撞逻辑

小恐龙和障碍物都抽象为矩形，当小恐龙和障碍物的矩形模型有交叠的部分时，判定小恐龙撞上了。但如果是按照原图片的像素大小来判断，会有四周的空位虚判而影响游戏体验，所以为了细化边界，我们采取了一个简单的方法：减小矩形的界限。

```
int CheckDinosaur(int x, int y, Plant plants[]) { //碰撞为1, 无碰撞为0
    for (int i = 0; i < PLANT_COUNT; i++) {
        if (x + Dinosaur_width - 14 > plants[i].x && x < plants[i].x + Plant_width - 14) //水平撞上了
            if (y + Dinosaur_height - 14 > plants[i].y) //竖直撞上了
                return 1; //碰撞的边界减小了矩形的界限
    }
    return 0; //无碰撞
}
```

图 23 小恐龙碰撞检测

3.5.7 记图变量

为了让各个区块工作的函数了解此时是处于哪个场景，我们定义了一个记图变量 `currentPlant`。它的取值是-1,0,1,2。从-1 开始的原因是因为第一个场景和第三个场景和第二个场景的逻辑 差别很大。我们为了表示方便，采用-1 开始的变量，这样在第二个场景的时候 `currentPlant` 的取值为 0，逻辑表达为 `false`，则记图变量可在一定判断下当作逻辑表达式。

```
if ((currentPlant && Check_Dinosaur(Dinosaur_X, Dinosaur_Y, plants) == 1)//植物碰撞
|| (!currentPlant && Check_Dinosaur_meteorites(Dinosaur_X, Dinosaur_Y, meteorites) == 1)) { //碰撞检测，进入分支说明发生碰撞
Show_Collision(Dinosaur_X, Dinosaur_Y, score, pause, ash, Bg_X, Bg_Y, Ground_X);//碰撞动画
Show_Game_Sketchpad(Bg_X, Bg_Y, BackGround_Img, Dinosaur_X, Dinosaur_Y, Dinosaur_Img,
currentPlant, plants, Plant_Img, Ground_X, Ground_Img, score, ash,
New_Plant, imgNew_Plant, meteorites, Meteorite_Img, Boom_Img, 2);//死亡状态
return;//游戏结束，退出程序
```

图 24 记图变量直接当作逻辑表达式的应用

3.5.8 小恐龙重力系统更新的实现

小恐龙不仅有坐标参数，还具有记录当前竖直方向加速度的变量 `Dinosaur_a`，其数值代表离开地面的时间长短，正代表加速度向上，负代表加速度向下。还有记录小恐龙是否在地面上的标记变量 `Dinosuar_flag`，用来判断小恐龙是否在地面，0 表示在地面，非 0 表示不在地面，这是优化时间的方法，不需要每一次起跳都判断小恐龙是否在地面上，优化程序运行效率。

下图为小恐龙的位置更新函数，加速度的应用体现：

```
void Move_Dinosaur(int& x, int& y, int& flag, int& a) {  
    y -= a * 3; //由加速度改变速度  
    //注意加速度是反过来的，正是往上，y是减小  
    a--;  
    //加速度肯定是一直往下的  
    if (y > face_height - Dinosaur_height - Ground_height) { //落地  
        a = 0;  
        flag = 0;  
        y = face_height - Dinosaur_height - Ground_height;  
    }  
}
```

图 25 记图变量直接当作逻辑表达式的应用